Monocular Depth Estimation

Autore: **Daniele Angeloni**
Università degli Studi di Perugia
Ingegneria Informatica e Robotica - Data Science and Data Engineering

***Abstract*—La *Monocular Depth Estimation* rappresenta una sfida complessa nella Computer Vision, in quanto richiede la predizione della distanza di ogni punto di una scena tridimensionale (3D) a partire da una singola immagine RGB (2D). In questo progetto è stata sviluppata una pipeline basata su deep learning supervisionato, progettando e addestrando un modello encoder-decoder per la predizione di mappe di profondità (depth-map). L'intero progetto è stato sviluppato utilizzando il framework PyTorch, che ha gestito il caricamento e la normalizzazione delle immagini RGB e delle mappe di profondità sintetiche, oltre alla definizione, addestramento e validazione del modello di deep learning. L'architettura utilizza una ResNet-50 pre-addestrata come encoder per l'estrazione di features multiscala e un decoder progettato manualmente, che integra upsampling bilineare e skip connections per la ricostruzione dei dettagli spaziali, ispirato dall'architettura di U-Net.**

La funzione di perdita combina errore quadratico medio (MSE) e indice di similarità strutturale (SSIM) per migliorare contemporaneamente precisione numerica e qualità percettiva. Sono state adottate strategie di training avanzate, per ottimizzare la convergenza e prevenire l'overfitting.

La valutazione finale sul set di test mostra un RMSE pari a 2.0527 e un SSIM pari a 0.6806, confermando la validità dell'architettura proposta e l'efficacia della pipeline di addestramento sviluppata.

I. Introduction

La capacità di stimare la profondità di una scena tridimensionale a partire da una sola immagine RGB, nota come *Monocular Depth Estimation*, rappresenta una sfida complessa. A differenza delle tecniche stereo o multi-view, che si basano su osservazioni multiple per inferire sulla geometria della scena, la stima monoculare è un problema altamente ambiguo dato che una singola proiezione bidimensionale può essere compatibile con molteplici configurazioni tridimensionali differenti.

Nonostante queste problematiche, gli sviluppi recenti nel deep learning hanno mostrato che è possibile apprendere, da grandi quantità di dati, correlazioni statistiche e strutturali tra caratteristiche visive e profondità della scena. In particolare, l'utilizzo di reti neurali profonde permette di combinare informazioni locali e globali per ottenere mappe di profondità coerenti e dettagliate.

All'interno di questo contesto si inserisce il progetto descritto in questo report, realizzato come parte di una challenge didattica proposta durante il corso universitario di Deep Learning and Robot Perception. Le specifiche della challenge richiedevano di:

- progettare un sistema capace di predire mappe di profondità da immagini RGB;
- utilizzare il dataset sintetico DepthEstimationUnreal, fornito per l'esercitazione;
- implementare la pipeline di addestramento, validazione e testing.

In risposta a tali requisiti, sono stati sviluppati diversi moduli software:

- `model.py`, per la definizione dell'architettura della rete neurale;
- `solver.py`, per la gestione delle procedure di training, validazione e test;
- `dataset.py`, per il caricamento strutturato delle immagini e delle depth map;
- `utils.py`, per il calcolo delle metriche di valutazione e la visualizzazione dei risultati;
- `main.py`, per la configurazione e l'esecuzione dei processi.

Il dataset utilizzato è il **DepthEstimationUnreal**, un insieme di immagini sintetiche che è stato fornito nell'ambito della challenge didattica, pensato per favorire un addestramento di tipo supervisionato con dimensioni **144×256** sia per le immagini RGB che per le mappe di profondità ground truth. Le immagini (memorizzate in formato JPEG) sono a 3 canali (RGB), mentre le depth map sono memorizzate come array NumPy, con valori reali compresi nel range $[0, 20]$ metri.

La struttura del dataset è organizzata come segue:

- `rgb/train` e `depth/train`: immagini RGB e depth map per il training;
- `rgb/val` e `depth/val`: dati riservati alla fase di validation
- `rgb/test` e `depth/test`: dati di test, lasciati vuoti e utilizzati esclusivamente dal docente per la valutazione finale.

L'obiettivo di questo lavoro è stato quindi quello di costruire una pipeline efficace e stabile con buoni risultati, capace di affrontare le principali difficoltà della Monocular Depth Estimation attraverso strategie di implementazione e scelte architetturali, di addestramento e di validazione, che siano motivate e consapevoli.

II. Related Work

Il problema della *Monocular Depth Estimation* ha ricevuto crescente attenzione nella comunità scientifica, poiché la capacità di inferire informazioni tridimensionali da una singola immagine 2D trova applicazione in numerosi ambiti pratici, come la navigazione autonoma, la robotica mobile e la realtà aumentata.

I primi tentativi di stima della profondità da immagini monoculari si basavano su tecniche geometriche tradizionali,

sfruttando indizi visivi come la prospettiva, le ombre ecc. Tuttavia, tali approcci risultavano altamente sensibili al contesto della scena e spesso inadatti a scenari complessi e/o dinamici.

Con l'avvento del deep learning, si è affermata una nuova generazione di metodi in grado di apprendere relazioni statistiche tra il contenuto visivo e la profondità della scena. I principali filoni di ricerca possono essere classificati come segue:

- **Approcci supervisionati**: basati sulla disponibilità di coppie immagine-profondità come ground-truth. Eigen et al. (2014) hanno introdotto un'architettura multiscala che predice mappe di profondità a diversi livelli di dettaglio, combinando una previsione globale con un affinamento locale [1]. Successivamente, architetture encoder-decoder con backbone pre-addestrati (es. ResNet, VGG) sono diventate lo standard per la regressione della profondità.
- **Approcci auto-supervisionati**: introdotti da Godard et al. (2017), eliminano la necessità di profondità ground-truth sfruttando la consistenza fotometrica tra immagini stereo o consecutive nel tempo. Questi metodi sono particolarmente utili in contesti dove i dati annotati sono difficili o costosi da ottenere [2].
- **Approcci multitasking**: mirano a migliorare la generalizzazione apprendendo simultaneamente compiti ausiliari, come la segmentazione semantica o la stima delle normali di superficie, condividendo la rappresentazione tra più task. Eigen e Fergus (2015) hanno proposto un'architettura multiscala comune per predire profondità, normali di superficie e etichette semantiche [3].

Ciascun approccio presenta vantaggi e limitazioni. I metodi supervisionati offrono generalmente maggiore precisione, ma richiedono dataset a disposizione di alta qualità. I metodi auto-supervisionati garantiscono scalabilità, a costo di una maggiore complessità nella definizione della funzione di perdita e di una minore accuratezza assoluta. Gli approcci multitasking cercano di bilanciare questi aspetti migliorando la robustezza del modello.

Nel contesto della challenge affrontata, si è optato per un approccio supervisionato tradizionale, in linea con le specifiche del progetto e il dataset a disposizione.

III. Proposed Approach

Il progetto implementa un modello per la *Monocular Depth Estimation* basato su un'architettura encoder-decoder, ispirata in parte alla struttura a "U" della U-Net [4] e adattata per sfruttare la potenza della ResNet-50 nella fase di codifica. L'obiettivo base è apprendere una funzione $f : \mathbb{R}^{3\times H\times W} \rightarrow \mathbb{R}^{1\times H'\times W'}$ che mappa un'immagine RGB tridimensionale in una mappa di profondità a singolo canale.

La progettazione dell'architettura e la gestione dei dati si basano sui principi visti a lezioni e ricercati in letteratura. In questa sezione vengono descritte in dettaglio le principali componenti implementate.

A. Model Architecture: model.py

La struttura segue un paradigma encoder-decoder, permettendo di estrarre rappresentazioni compatte dell'immagine per poi ricostruire la mappa di profondità con risoluzione piena.

1) Encoder: Feature Extraction con ResNet-50: Il blocco encoder è basato su una **ResNet-50** [5] pre-addestrata su ImageNet, importata tramite il modulo `torchvision.models`. L'adozione di questa rete rappresenta un tipico esempio di *transfer learning*, in cui si sfruttano le conoscenze acquisite da un modello addestrato su un ampio dataset per un nuovo task.

Nel contesto del progetto è stato adottato un approccio basato sul *transfer learning* con *fine-tuning*: è stata utilizzata la ResNet-50 preaddestrata su ImageNet come encoder, mantenendone i layer convoluzionali e inizializzandoli con i pesi già ottimizzati. Sono stati invece rimossi sia il *global average pooling* che il classificatore finale *fully connected*, non compatibili con il task di regressione spaziale richiesto dalla stima della profondità. Durante l'addestramento, i pesi dei layer convoluzionali sono stati ulteriormente raffinati tramite backpropagation sul dataset di riferimento, consentendo al modello di adattare le rappresentazioni apprese al nuovo dominio. Questa strategia ha permesso di ridurre significativamente il tempo di convergenza e di migliorare la stabilità e la capacità di generalizzazione del modello.

I moduli della ResNet-50 utilizzati nella fase di codifica del modello corrispondono ai principali blocchi convoluzionali dell'architettura originale, mantenuti fino a `layer4`. In particolare:

- `conv1, bn1, relu, maxpool`: costituiscono lo *stem iniziale* e si occupano di estrarre pattern visivi di basso livello, come corner e edge, utili in diversi contesti visivi.
- `layer1` (256 canali): contiene *3 bottleneck block* con skip connection, e cattura texture e forme locali fornendo una prima astrazione della scena.
- `layer2` (512 canali): composto da *4 bottleneck block* con downsampling, individua contorni e separazioni tra superfici, mantenendo stabilità nel gradiente grazie alle connessioni residue.
- `layer3` (1024 canali): con *6 bottleneck block*, modella relazioni spaziali più ampie, fondamentali per comprendere la struttura globale della scena.
- `layer4` (2048 canali): formato da *3 bottleneck block*, genera feature astratte e semantiche, ideali per alimentare la fase di decodifica nella ricostruzione della mappa di profondità.

Durante il forward pass, ciascuna di queste feature map viene salvata e successivamente riutilizzata nel decoder tramite *skip connections*, consentendo di recuperare informazioni spaziali dettagliate che verrebbero altrimenti perse nei successivi downsampling.

Il fine-tuning di questi blocchi tramite backpropagation permette al modello di adattare le rappresentazioni pre-apprese al dominio specifico del dataset sintetico, garantendo al contempo un buon equilibrio tra accuratezza, generalizzazione e costo computazionale. Quindi l'intera parte convoluzionale della ResNet-50 è stata riutilizzata come encoder, mentre i layer finali dedicati alla classificazione (`avgpool` e `fc`) sono stati rimossi. Al loro posto è stato progettato un decoder specifico per ricostruire la mappa di profondità, mantenendo

l'output a singolo canale coerente con il task di *Monocular Depth Estimation*.

2) Decoder: Ricostruzione Progressiva e Skip Connections: Il decoder è stato progettato manualmente per ricostruire progressivamente la mappa di profondità, incrementando la risoluzione spaziale ad ogni step.

Ad ogni livello di decodifica viene eseguita:

- un'**interpolazione bilineare** utilizzata per eseguire l'upsampling, con l'obiettivo di raddoppiare la risoluzione spaziale delle feature map
- una **concatenazione** della feature upsampled con la corrispondente feature map salvata dall'encoder;
- una **convoluzione 2D** (`torch.nn.Conv2d`) seguita da Batch Normalization (`torch.nn.BatchNorm2d`) e funzione di attivazione ReLU (`torch.nn.ReLU`).

La sequenza dei moduli di decodifica è:

- `conv1`: decodifica da 2048 a 1024 canali;
- `conv2`: 1024 + 1024 concatenati $\rightarrow$ 512 canali;
- `conv3`: 512 + 512 concatenati $\rightarrow$ 256 canali;
- `conv4`: 256 + 256 concatenati $\rightarrow$ 128 canali;
- `conv5`: upsampling e convoluzione 128 $\rightarrow$ 64 canali;
- `conv6`: convoluzione finale 1x1 per ottenere una profondità a singolo canale.

Il decoder produce una mappa più piccola e la ingrandisce a 144×256 pixel usando interpolazione bilineare, per ottenere un risultato finale con la dimensione desiderata.

B. Output Depth Map

L'output del modello è una mappa a singolo canale che associa ad ogni pixel dell'immagine RGB in input un valore di profondità espresso in metri. In fase di training, i valori di profondità sono vincolati all'interno dell'intervallo $[0, 20]$ metri, coerentemente con il range del dataset DepthEstimationUnreal.

C. Dataset Management: dataset.py

Il caricamento e la gestione del dataset sono gestiti dalla classe `DepthDataset`, che estende l'interfaccia standard `torch.utils.data.Dataset`.

Le funzionalità principali includono:

- Lettura delle immagini RGB tramite `torchvision.io.read_image`, con successiva normalizzazione dei pixel in $[0, 1]$.
- Caricamento delle mappe di profondità da file `.npy` (NumPy), conversione in tensori PyTorch, aggiunta di una dimensione di canale e limitazione dei valori nel range tra 0 e 20.
- Selezione dinamica della modalità `train`, `val` o `test` in base alla fase di utilizzo.

Sono state applicate operazioni di shuffle tra ogni fase mentre non è stata applicata data augmentation, poiché il dataset sintetico presenta già un'adeguata variabilità visiva.

D. Loss Function: MSE + SSIM

La funzione di perdita scelta combina due obiettivi differenti:

$$\mathcal{L}(D, \hat{D}) = \alpha \cdot \text{MSE}(D, \hat{D}) + (1 - \alpha) \cdot (1 - \text{SSIM}(D, \hat{D}))$$

- Il termine MSE (Mean Square Error) (`torch.nn.functional.mse_loss`) misura la differenza quadratica media tra i valori predetti e il ground-truth, penalizzando gli errori numerici locali.
- Il termine SSIM [6] (Structural Similarity Index Measure) (`utils.ssim`) valuta la similarità strutturale tra le due immagini, favorendo il mantenimento di bordi e dettagli geometrici.

Questa combinazione pesata è stata scelta per migliorare sia la precisione numerica assoluta che la coerenza percettiva delle mappe di profondità generate.

E. Scelte progettuali e motivazioni

Le principali scelte progettuali sono motivate da:

- **Utilizzo di ResNet-50 pre-addestrata**: per trasferire conoscenza visiva generalizzata e accelerare la fase di training.
- **Skip connections nel decoder**: per ripristinare dettagli spaziali persi nei downsampling.
- **Funzione di Loss ibrida MSE + SSIM**: per ottenere mappe di profondità realistiche e fedeli alla geometria della scena.
- **Upsampling bilineare**: per aumentare la risoluzione delle feature map in modo regolare e senza introdurre artefatti visivi, come quelli tipicamente causati dalle transpose convolutions (effetto scacchiera).

Tali scelte sono state adottate per massimizzare l'efficacia del modello, garantendo un buon compromesso tra accuratezza, stabilità numerica e semplicità implementativa.

IV. Experiments

In questa sezione vengono descritte nel dettaglio le scelte sperimentali adottate per l'addestramento, la validazione e la valutazione finale del modello di *Monocular Depth Estimation*, oltre a quelle già descritte nella sezione precedente.

A. Inizializzazione (Training Setup)

L'addestramento è stato realizzato utilizzando le seguenti impostazioni principali:

- **Ottimizzatore**: è stato utilizzato Adam, scelto per la sua capacità di adattare dinamicamente il learning rate *a livello di singolo parametro*, combinando i vantaggi del momentum e della normalizzazione del gradiente. Questa caratteristica consente aggiornamenti più stabili e veloci durante il training, soprattutto in reti profonde. [7];
- **Learning rate iniziale**: 5×10^{-4}, con **weight decay** pari a 10^{-5}, utilizzato come regolarizzazione L2 per penalizzare pesi troppo grandi.
- **Batch size**: 32, scelta come compromesso tra stabilità numerica e velocità computazionale (limitato da utilizzo CPU);

- **Scheduler**: `ReduceLROnPlateau`, utilizzato per ridurre automaticamente la learning rate quando la loss di validation non mostra miglioramenti per un numero definito di epoche consecutive, favorendo così una convergenza più stabile nelle fasi avanzate del training.
- **Early stopping**: implementato con una *patience* di 10 epoche, consente di interrompere anticipatamente l'addestramento in caso di non miglioramento delle prestazioni, riducendo il rischio di andare in overfitting.

Inoltre, durante il training, è stata adottata una strategia di **tuning manuale della funzione di perdita**: i pesi tra i termini MSE e SSIM sono stati modificati dinamicamente sulla base dell'osservazione empirica delle curve di training e validazione e dei valori parziali di SSIM e RMSE, al fine di ottimizzare il bilanciamento tra accuratezza numerica e qualità strutturale delle mappe predette.

B. Gestione del Training in più Sessioni

A causa della mancanza di una GPU dedicata, il training è stato condotto in più sessioni separate. Per gestire questa necessità, sono stati implementati:

- Il salvataggio automatico dei checkpoint dopo ogni epoca (`torch.save`);
- La ripresa del training da un checkpoint esistente (`torch.load`).

In alcune fasi, per accelerare il processo, si è fatto uso di "Google Colab", compatibilmente con le limitazioni temporali del piano gratuito fornito.

Complessivamente, sono state completate più di **70 epoche** di training effettivo, attraverso sessioni su dispositivi locali e cloud.

Questa modalità ha permesso di superare i limiti hardware disponibili, garantendo la continuità dell'addestramento anche su dispositivi diversi. Nonostante l'esecuzione frammentata, il training ha mantenuto coerenza e stabilità, grazie all'uso dei checkpoint. La suddivisione in sessioni non ha compromesso la qualità del modello, che ha prodotto mappe di profondità accurate e visivamente consistenti.

C. Metriche di Valutazione

Le prestazioni del modello sono valutate tramite:

- **Root Mean Squared Error (RMSE)**:

$$\text{RMSE} = \sqrt{\frac{1}{N}\sum_{i=1}^{N}(\hat{D}_i - D_i)^2}$$

  dove $\hat{D}$ è la profondità predetta e D quella reale.
- **Structural Similarity Index Measure (SSIM)**:

$$\text{SSIM}(\hat{D}, D) = \frac{(2\mu_{\hat{D}}\mu_D + C_1)(2\sigma_{\hat{D}D} + C_2)}{(\mu_{\hat{D}}^2 + \mu_D^2 + C_1)(\sigma_{\hat{D}}^2 + \sigma_D^2 + C_2)}$$

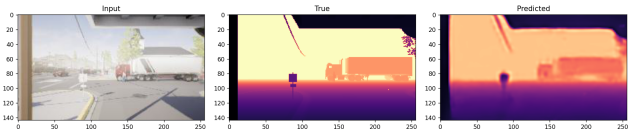
Il RMSE misura l'errore assoluto medio, mentre il SSIM valuta la qualità strutturale e percettiva della predizione.

D. Visualizzazione dei Risultati

Durante l'addestramento e la validazione, ogni N batch (tipicamente ogni 50), viene generata una figura tramite una funzione fornita in `utils.py`, contenente:

- L'immagine RGB di input;
- La mappa di profondità ground-truth;
- La mappa di profondità predetta.

Questa visualizzazione permette un'interpretazione qualitativa delle predizioni.



In parallelo, vengono stampati a schermo anche i valori delle metriche quantitative, tra cui l'`RMSE` e l'`SSIM`, oltre alla loss sul training e sul validation e alla learning rate corrente. Queste informazioni sono fondamentali per monitorare costantemente l'andamento dell'apprendimento del modello, individuare eventuali casi di overfitting e guidare le strategie di ottimizzazione.

E. Osservazioni sperimentali

Durante il processo di addestramento sono emerse diverse osservazioni rilevanti:

- L'integrazione del termine SSIM nella funzione di perdita ha portato a una migliore nitidezza dei bordi e a una maggiore coerenza strutturale delle mappe di profondità rispetto all'uso esclusivo del termine MSE;
- Le skip connections tra encoder e decoder si sono dimostrate fondamentali per preservare i dettagli spaziali a diverse scale di risoluzione;
- L'adozione del learning rate scheduler `ReduceLROnPlateau` ha favorito la stabilità della discesa della loss nelle fasi finali di training, consentendo un monitoraggio efficace e una gestione dinamica del learning rate;
- L'ottimizzatore Adam, combinato con un termine di weight decay, ha garantito aggiornamenti dei pesi adattivi ed efficaci, contribuendo alla riduzione del rischio di overfitting senza compromettere la capacità di apprendimento;

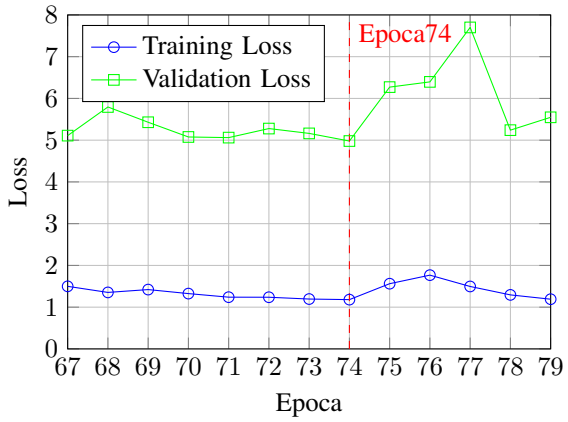
F. Checkpoint Migliore in Validazione

Il miglior checkpoint, salvato all'epoca 74, ha ottenuto i seguenti risultati in validazione:

- **RMSE**: 2.5773
- **SSIM**: 0.5310
- **Validation Loss**: 4.9794
- **Training Loss**: 1.1804
- **Learning Rate**: 0.0005

Il modello è stato salvato come:

`./checkpoint\depth_74.pth`

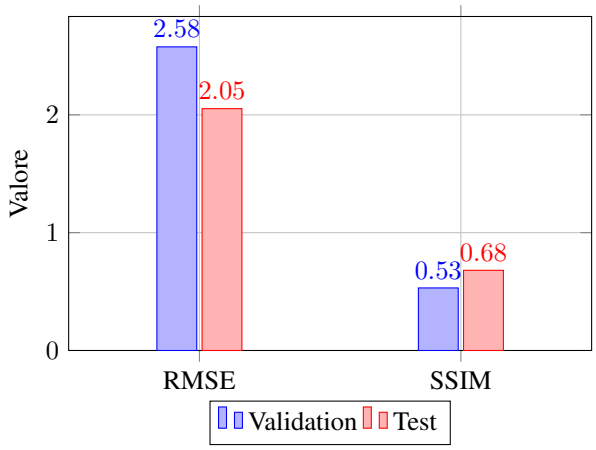


Andamento della Training Loss e Validation Loss tra le ultime epoche di addestramento e scelta del checkpoint migliore (Epoca 74)

G. *Test finale (Risultati Challenge)*

Il checkpoint selezionato è stato inviato per la valutazione ufficiale sul set di test non accessibile. I risultati finali ottenuti sono:

- **RMSE**: 2.0527
- **SSIM**: 0.6806



Confronto dei risultati Validation e Test: RMSE e SSIM.

Questi valori confermano la capacità del modello di generalizzare su dati non visti, dimostrando l'efficacia della strategia architetturale e della pipeline di addestramento adottata. Si osserva infatti che SSIM è migliorato grazie ad un aumento del suo valore, mentre l'RMSE è migliorato grazie a una sua diminuzione, indicando rispettivamente una migliore qualità strutturale e una minore differenza media quadratica tra predizione e realtà.

V. Conclusions

L'architettura proposta, composta da un encoder ResNet-50 pre-addestrato e ottimizzato tramite fine-tuning, e da un decoder progettato manualmente con skip connection e upsampling bilineare, si è dimostrata efficace nella generazione di mappe di profondità coerenti sia a livello numerico che percettivo.

L'addestramento condotto su un dataset sintetico di alta qualità (DepthEstimationUnreal) ha beneficiato dell'integrazione di varie tecnologie, tra cui l'*early stopping*, la schedulazione dinamica del learning rate mediante `ReduceLROnPlateau`, Adam come ottimizzatore e una funzione di perdita composita che combina errore quadratico medio (MSE) e indice di similarità strutturale (SSIM). La gestione dei checkpoint ha inoltre consentito di suddividere il training in più sessioni, superando i vincoli hardware.

I risultati ottenuti sul set di test — con un RMSE pari a 2.0527 e un SSIM pari a 0.6806 — confermano la robustezza dell'architettura proposta. Non sono emersi segnali di overfitting o underfitting, come dimostrato dalla coerenza tra le performance su training e validazione.

Un dato particolarmente positivo è rappresentato dalle prestazioni sul test, superiori a quelle ottenute in validazione. Questo potrebbe essere dovuto sia a una buona capacità di generalizzazione favorita dalle strategie di training adottate, sia a una distribuzione dei dati di test più regolare o meno rumorosa.

Anche l'analisi qualitativa ha mostrato risultati convincenti: l'integrazione del termine SSIM nella funzione di perdita ha portato a predizioni più nitide e strutturalmente coerenti, dimostrandosi determinante per la qualità visiva delle depth map generate.

Per sviluppi futuri, il progetto potrebbe essere esteso con l'introduzione di tecniche di data augmentation, al fine di aumentare la variabilità dei dati e migliorare ulteriormente la generalizzazione. Un'altra direzione promettente è rappresentata dall'integrazione di architetture basate su Transformer, che stanno ottenendo risultati competitivi anche in ambito visivo.

In sintesi, questo lavoro ha confermato l'efficacia delle soluzioni architetturali e metodologiche adottate, offrendo al tempo stesso un'opportunità concreta di applicare e consolidare i concetti appresi nel corso, in un contesto progettuale completo e stimolante.

References

[1] D. Eigen, C. Puhrsch, and R. Fergus, "Depth map prediction from a single image using a multi-scale deep network," *Advances in neural information processing systems*, vol. 27, 2014. [Online]. Available: https://arxiv.org/abs/1406.2283

[2] C. Godard, O. Mac Aodha, and G. J. Brostow, "Unsupervised monocular depth estimation with left-right consistency," in *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2017, pp. 270–279. [Online]. Available: https://arxiv.org/abs/1609.03677

[3] D. Eigen and R. Fergus, "Predicting depth, surface normals and semantic labels with a common multi-scale convolutional architecture," in *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 2650–2658. [Online]. Available: https://arxiv.org/abs/1411.4734

[4] GeeksforGeeks, "U-net architecture - explained," https://www.geeksforgeeks.org/u-net-architecture-explained/, 2023.

[5] K. He, X. Zhang, S. Ren, and J. Sun, "Deep residual learning for image recognition," *Proceedings of the IEEE conference on computer vision and pattern recognition*, pp. 770–778, 2016. [Online]. Available: https://arxiv.org/abs/1512.03385

[6] Z. Wang, A. C. Bovik, H. R. Sheikh, and E. P. Simoncelli, "Image quality assessment: From error visibility to structural similarity," *IEEE transactions on image processing*, vol. 13, no. 4, pp. 600–612, 2004.

[7] D. P. Kingma and J. Ba, "Adam: A method for stochastic optimization," *arXiv preprint arXiv:1412.6980*, 2014. [Online]. Available: https://arxiv.org/abs/1412.6980